

HLD

High-Level Design

System architecture, components, and integration model

Document	HLD
Version	1.0
Status	Approved
Owner	Swiggy Nexus AI Team
Audience	Engineering, Product, Design, Reviewers
Last Updated	July 2026

This document is part of the Swiggy Nexus AI reference implementation, an AI-powered personal commerce assistant built on the Swiggy MCP. It captures decisions, requirements, and design intent for engineers, reviewers, and stakeholders.

1. Overview

Nexus AI is a modular, event-driven system composed of a Next.js frontend, a FastAPI backend, a LangGraph agent runtime, a typed Swiggy MCP client, a memory subsystem backed by PostgreSQL + pgvector, and a Celery worker pool for asynchronous jobs.

2. Architecture at a Glance



Figure 1. High-level component topology.

3. Component Responsibilities

Component	Responsibility
Frontend (Next.js)	Server-rendered UI, streaming chat, cart preview, analytics.
API Gateway (FastAPI)	Auth, rate-limit, SSE, request validation, orchestration entry point.
Agent Runtime	LangGraph state machine: route → plan → act → observe → synthesize.
MCP Client	Typed wrapper over Swiggy MCP; retries, circuit breaker, tracing.
Memory Service	Short-term (Redis), long-term (pgvector), summarization jobs.
Workers (Celery)	Order polling, notification dispatch, embeddings, digests.
Datastore	PostgreSQL 16 with pgvector; primary system of record.
Cache/Queue	Redis 7; response cache, rate-limit tokens, Celery broker.
Observability	OpenTelemetry → Tempo, Prometheus, Loki, Grafana.

4. Deployment View

- Kubernetes (EKS, ap-south-1) with separate node pools for API, workers, and system add-ons.
- Cloudflare in front for CDN + WAF; Traefik ingress inside the cluster.
- Blue/green rollouts; canary via header-based routing.
- Postgres via managed RDS (Multi-AZ, PITR); Redis via ElastiCache.

5. Data Flow (Chat → Order)

- 1. Client opens SSE stream to /v1/chat.
- 2. API loads user context and memory summary.

- 3. Agent plans and fans out MCP tool calls in parallel where safe.
- 4. Cart draft persisted; preview streamed to client.
- 5. User confirms; agent invokes place_order.
- 6. Order + episodic memory persisted; webhooks update tracking.

6. Cross-Cutting Concerns

Concern	Approach
Security	OAuth 2.1 + PKCE, KMS envelope encryption, audit log.
Reliability	Idempotent order APIs, circuit breakers, dead-letter queues.
Observability	Every MCP call is a span; per-request cost + tokens attributed.
Cost Control	Router model, response cache, hard per-user budget.
Compliance	DPDP Act, PCI-DSS scope reduction, retention policies enforced in DB.

7. Capacity Planning

Metric	Assumption	Provisioned
Peak QPS (chat)	300	500 headroom
Peak QPS (MCP)	1,500 (5× fan-out)	2,500
LLM tokens / order	~4,500	Budget cap 8,000
Storage / user / yr	~120 MB	200 MB soft cap

8. Failure Modes

- LLM outage → fallback provider + degraded template responses.
- MCP tool outage → circuit-break for 30s; explain to user; suggest retry.
- DB failover → transparent via RDS Multi-AZ; workers pause writes for 15s.
- Cache miss storm → request coalescing + jittered backfill.